4_MACHINE_LEARNING_PIPELINE

🤖 MACHINE LEARNING PIPELINE FOR MENTAL HEALTH PREDICTION

Complete Preprocessing, Modeling & Prediction Workflow

🎯 PROJECT OBJECTIVES

Prediction Goals:

1. **Risk Classification**: Predict Low/Medium/High mental health risk
2. **Treatment Seeking**: Predict likelihood of seeking mental health treatment
3. **Symptom Severity**: Predict depression/anxiety score levels
4. **Intervention Response**: Predict response to specific interventions

Business Applications:

- Early intervention targeting
- Resource allocation optimization
- Personalized treatment recommendations
- Population health monitoring

📊 DATA PREPROCESSING PIPELINE

Phase 1: Data Cleaning & Preparation

1.1 Data Quality Assessment

Python

```python
# Initial data inspection
df.info()  # Check data types and missing values
df.describe()  # Statistical summary
df.isnull().sum()  # Missing value verification
```

Findings:

- 0 missing values across all 14 variables
- All numeric variables within expected ranges
- Categorical variables properly encoded

1.2 Feature Engineering

Python

```python
# Create composite risk scores
df['risk_composite'] = (df['depression_score'] * 0.6) +
(df['anxiety_score'] * 0.4)

# Derive categorical age groups
df['age_group'] = pd.cut(df['age'], bins=[18, 30, 50, 65],
                         labels=['Young Adult', 'Middle Adult', 'Senior'])

# Create stress-sleep interaction term
df['stress_sleep_interaction'] = df['stress_level'] *
(1/df['sleep_hours'])
```

1.3 Encoding Categorical Variables

Python

```python
from sklearn.preprocessing import LabelEncoder, OneHotEncoder

# Binary encoding for Yes/No variables
binary_vars = ['mental_health_history', 'seeks_treatment']
for var in binary_vars:
    le = LabelEncoder()
    df[var] = le.fit_transform(df[var])  # Yes=1, No=0

# One-hot encoding for multi-category variables
categorical_vars = ['gender', 'employment_status', 'work_environment']
df_encoded = pd.get_dummies(df, columns=categorical_vars,
drop_first=True)
```

1.4 Handling Class Imbalance

Python

```python
from imblearn.over_sampling import SMOTE

# Address risk category imbalance
X = df_encoded.drop(['mental_health_risk'], axis=1)
y = df_encoded['mental_health_risk']

smote = SMOTE(random_state=42)
X_balanced, y_balanced = smote.fit_resample(X, y)
```

🧪 FEATURE SELECTION & ENGINEERING

2.1 Correlation Analysis

Python

```python
# Feature correlation matrix
correlation_matrix = df_encoded.corr()
high_corr_features = correlation_matrix[abs(correlation_matrix['mental_health_risk']) > 0.3].index.tolist()

# Remove highly correlated features to prevent multicollinearity
selected_features = [
    'depression_score', 'anxiety_score', 'stress_level', 'sleep_hours',
    'social_support_score', 'physical_activity_days', 'age',
    'gender_Female', 'employment_status_Employed'
]
```

2.2 Feature Scaling

Python

```python
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()
X_scaled = scaler.fit_transform(X[selected_features])
```

🤖 MODEL DEVELOPMENT PIPELINE

3.1 Model Selection Framework

Primary Models to Evaluate:

1. **Decision Tree Classifier** - Simple, interpretable, covered in curriculum
2. **Logistic Regression** - Linear model, well-understood, covered in curriculum
3. **K-Nearest Neighbors (KNN)** - Instance-based learning, covered in curriculum
4. **Support Vector Machine (SVM)** - Margin-based classification, covered in curriculum
5. **Random Forest Classifier** - Ensemble method (if time permits)

Evaluation Metrics:

- **Accuracy**: Overall correct predictions
- **Precision/Recall**: Class-specific performance
- **F1-Score**: Balance between precision and recall
- **AUC-ROC**: Discrimination ability
- **Confusion Matrix**: Detailed error analysis

3.2 Cross-Validation Strategy

Python

```python
from sklearn.model_selection import StratifiedKFold

# 5-fold stratified cross-validation
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
```

📈 MODEL TRAINING & VALIDATION

4.1 Training Process

Python

```python
from sklearn.tree import DecisionTreeClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.neighbors import KNeighborsClassifier
from sklearn.svm import SVC
from sklearn.metrics import classification_report, confusion_matrix

# Split data
X_train, X_test, y_train, y_test = train_test_split(
```

```python
    X_scaled, y_balanced, test_size=0.2, random_state=42,
stratify=y_balanced
)

# Train Multiple Models
models = {
    'Decision Tree': DecisionTreeClassifier(random_state=42),
    'Logistic Regression': LogisticRegression(random_state=42),
    'KNN': KNeighborsClassifier(n_neighbors=5),
    'SVM': SVC(probability=True, random_state=42)
}

trained_models = {}
for name, model in models.items():
    model.fit(X_train, y_train)
    trained_models[name] = model
    print(f"{name} trained successfully")
```

4.2 Model Evaluation

Python

```python
# Evaluate all models
results = {}
for name, model in trained_models.items():
    y_pred = model.predict(X_test)
    y_pred_proba = model.predict_proba(X_test)

    accuracy = model.score(X_test, y_test)
    auc_score = roc_auc_score(y_test, y_pred_proba, multi_class='ovr')

    results[name] = {
        'accuracy': accuracy,
        'auc_roc': auc_score,
        'classification_report': classification_report(y_test, y_pred)
    }

    print(f"\n{name} Results:")
    print(f"Accuracy: {accuracy:.3f}")
    print(f"AUC-ROC: {auc_score:.3f}")
    print(classification_report(y_test, y_pred))

# Select best model based on validation
best_model_name = max(results.keys(), key=lambda x: results[x]
['accuracy'])
best_model = trained_models[best_model_name]
```

```
print(f"\nBest Model: {best_model_name} with accuracy {results[best_model_name]['accuracy']:.3f}")
```

🔮 PREDICTION WORKFLOW & EXAMPLES

5.1 Real-World Prediction Scenarios

Example 1: Individual Risk Assessment

Input Profile:

```
Age: 28
Gender: Female
Employment: Employed
Work Environment: Remote
Mental Health History: No
Seeks Treatment: No
Stress Level: 8
Sleep Hours: 5.5
Physical Activity: 2 days/week
Depression Score: 22
Anxiety Score: 16
Social Support: 45
Productivity: 65
```

Processing Pipeline:

1. **Feature Engineering**: Calculate composite scores and interactions
2. **Encoding**: Convert categorical variables to numerical
3. **Scaling**: Normalize features to training data scale
4. **Prediction**: Apply trained model

Output:

```
Predicted Risk: HIGH (85% probability)
Recommended Action: Immediate intervention suggested
Confidence: Strong (probability > 80%)
```

Logical Reasoning:

- High depression (22) and anxiety (16) scores trigger high risk
- Poor sleep (5.5 hours) amplifies risk
- Low social support (45) removes protective factors
- High stress (8) without treatment history indicates urgency

Example 2: Treatment Seeking Prediction

Input Profile:

```
Age: 45
Gender: Male
Employment: Self-employed
Work Environment: Hybrid
Mental Health History: Yes
Seeks Treatment: No
Stress Level: 6
Sleep Hours: 7.0
Physical Activity: 4 days/week
Depression Score: 18
Anxiety Score: 12
Social Support: 75
Productivity: 78
```

Processing Pipeline:

1. **Feature Selection**: Focus on treatment-seeking predictors
2. **Model Application**: Use treatment-seeking specific model
3. **Probability Calculation**: Estimate likelihood of future treatment seeking

Output:

```
Treatment Seeking Probability: 25%
Barriers Identified:
- Cost concerns (high self-employed population)
- Stigma (male demographic pattern)
- Satisfaction with current coping (moderate scores)

Intervention Strategy:
- Financial assistance programs
- Male-focused mental health messaging
- Peer support group introduction
```

Logical Reasoning:

- Male gender historically shows lower treatment rates
- Self-employed status correlates with financial barriers
- Moderate symptom levels suggest ambivalence about treatment
- Good social support indicates potential peer influence leverage

Example 3: Workplace Mental Health Screening

Input Profile:

```
Age: 35
Gender: Non-binary
Employment: Employed
Work Environment: On-site
Mental Health History: No
Seeks Treatment: No
Stress Level: 9
Sleep Hours: 4.8
Physical Activity: 1 day/week
Depression Score: 25
Anxiety Score: 19
Social Support: 30
Productivity: 58
```

Processing Pipeline:

1. **Risk Stratification**: Multi-model ensemble approach
2. **Workplace Context**: Adjust for occupational stress factors
3. **Urgency Scoring**: Calculate immediate intervention need

Output:

```
Risk Classification: CRITICAL HIGH
Intervention Priority: IMMEDIATE (within 48 hours)
Recommended Actions:
1. Manager notification for wellness check
2. Employee Assistance Program referral
3. Flexible work arrangement consideration
4. Crisis intervention resource provision

Risk Factors:
- Extremely high stress (9/10)
- Severe sleep deprivation (4.8 hours)
- Critical productivity impact (58%)
- Minimal social support (30)
```

Logical Reasoning:

- Non-binary individuals show elevated risk patterns in dataset
- On-site work environment may contribute to stress
- Extremely poor sleep indicates severe impairment
- Productivity decline suggests functional impact
- Absence of treatment history with severe symptoms indicates urgent need

---

🛠️ IMPLEMENTATION ARCHITECTURE

6.1 Production Pipeline Structure

API Endpoint Design:

Python

```python
@app.route('/predict_mental_health_risk', methods=['POST'])
def predict_risk():
    # 1. Receive JSON input
    input_data = request.json

    # 2. Preprocess input
    processed_data = preprocess_input(input_data)

    # 3. Generate prediction
    prediction = model.predict(processed_data)
    probability = model.predict_proba(processed_data)

    # 4. Return structured response
    return jsonify({
        'risk_level': prediction[0],
        'confidence': max(probability[0]),
        'recommendations': generate_recommendations(prediction[0])
    })
```

Batch Processing System:

Python

```python
def batch_risk_assessment(employee_data):
    """
    Process large datasets for organizational screening
    """
    # Preprocess batch data
    processed_batch = preprocess_dataframe(employee_data)

    # Generate predictions
    predictions = model.predict(processed_batch)
    probabilities = model.predict_proba(processed_batch)

    # Create risk stratification report
    return generate_stratification_report(predictions, probabilities)
```

📊 MONITORING & MAINTENANCE

7.1 Model Performance Tracking

Python

```python
# Continuous monitoring metrics
performance_metrics = {
    'accuracy_trend': [],
    'precision_by_class': [],
    'recall_by_class': [],
    'feature_drift': [],
    'data_quality_scores': []
}

def monitor_model_performance(new_predictions, actual_outcomes):
    """Track model performance over time"""
    # Calculate current metrics
    current_accuracy = accuracy_score(actual_outcomes, new_predictions)

    # Detect performance degradation
    if current_accuracy < (baseline_accuracy - 0.05):
        trigger_model_retraining()
```

7.2 Feedback Loop Integration

Python

```python
def incorporate_feedback(feedback_data):
    """Integrate user feedback for model improvement"""
    # Add feedback to training dataset
    updated_training_data = combine_datasets(training_data,
feedback_data)

    # Retrain model with enhanced data
    retrain_model(updated_training_data)
```

🔒 ETHICAL CONSIDERATIONS & SAFETY

8.1 Privacy Protection

- **Data Anonymization**: Remove personally identifiable information
- **Consent Management**: Explicit opt-in for predictions

- **Access Controls**: Role-based permission systems
- **Audit Trails**: Log all prediction activities

8.2 Bias Mitigation

Python

```python
def check_prediction_bias(predictions, demographic_data):
    """Ensure fair treatment across demographic groups"""
    bias_metrics = {}

    for demographic in ['gender', 'age_group']:
        bias_metrics[demographic] = calculate_demographic_parity(
            predictions, demographic_data[demographic]
        )

    return bias_metrics
```

8.3 Safety Protocols

- **Crisis Detection**: Automatic escalation for high-risk predictions
- **Human Oversight**: Review required for critical decisions
- **Uncertainty Quantification**: Confidence intervals for all predictions
- **Fallback Procedures**: Manual review when automated predictions uncertain

---

🚀 DEPLOYMENT SCENARIOS

9.1 Healthcare Settings

- **Primary Care Integration**: EHR system alerts
- **Specialty Referrals**: Automated psychiatrist matching
- **Population Health**: Community risk monitoring

9.2 Workplace Applications

- **Employee Wellness**: Proactive mental health screening
- **Manager Training**: Risk identification tools
- **Benefits Optimization**: Targeted mental health resources

9.3 Educational Institutions

- **Student Support**: Early intervention systems
- **Counseling Services**: Resource allocation optimization
- **Academic Success**: Mental health-academic performance linkage

---

This comprehensive machine learning pipeline transforms mental health data into actionable predictions while maintaining ethical standards and clinical validity